BILLBUDDY / PAISATRACK

Complete Project Documentation

Project Title: BillBuddy / PaisaTrack — Expense Tracker and Split Bill Management System

Developer: PRAYAG SHARMA

Project Type: Full-Stack Web Application

Project Category: Personal Finance Management and Expense Sharing

Frontend Technologies: HTML5, CSS3, JavaScript

Backend Technology: Java with Spring Boot

Database: MySQL

Database Integration: Spring Data JPA and Hibernate

Build and Dependency Management: Apache Maven

Development Environment: Visual Studio Code

---

1. PROJECT INTRODUCTION

BillBuddy, also referred to as PaisaTrack during development, is a personal expense tracking and bill-sharing web application designed to help users manage their daily financial transactions and divide shared expenses among friends.

Managing personal finances can become difficult when a person has multiple sources of income, makes frequent purchases, shares expenses with friends, or participates in group activities. People often forget how much they have spent, struggle to calculate their share of a bill, and find it difficult to remember who owes money to whom.

Traditional methods of tracking expenses, such as maintaining handwritten records, using calculators, or remembering payments mentally, can be inconvenient and prone to errors.

BillBuddy/PaisaTrack addresses these problems by providing a centralized digital platform for recording income and expenses, reviewing spending patterns, managing shared bills, and tracking settlements between individuals.

The application combines two major functionalities:

1. **Personal Expense Management:** Recording and monitoring individual income and expenses.
2. **Split Bill Management:** Dividing shared expenses, tracking individual shares, and maintaining records of settlements.

The project was initially developed as a frontend-based web application using HTML, CSS, and JavaScript. It was subsequently expanded toward a full-stack architecture using Java Spring Boot and MySQL to support backend processing and persistent database storage.

The primary objective is to create a practical, user-friendly, and reliable financial management application suitable for everyday use and academic project demonstration.

2. PROBLEM STATEMENT

People frequently encounter difficulties when managing their personal finances and sharing expenses with others.

Some of the most common problems include:

- Difficulty maintaining an accurate record of daily expenses.
- Lack of a clear overview of total income and expenditure.
- Difficulty identifying where money is being spent.
- Forgetting previously recorded transactions.
- Complications when editing or correcting expense records.
- Difficulty dividing restaurant bills, travel expenses, shopping bills, and other shared costs.
- Confusion about how much each participant owes.
- Disagreements regarding payments that have already been made.
- Difficulty maintaining a history of settlements between friends.
- Lack of a single place to manage personal transactions and shared expenses.

BillBuddy/PaisaTrack aims to address these problems through an organized digital interface that allows users to manage personal transactions and shared financial obligations.

The application reduces manual calculations, improves record organization, and makes expense-sharing information easier to understand.

---

3. PROJECT OBJECTIVES

The main objectives of the project are as follows.

3.1 Personal Financial Management

To develop a web application that allows users to record income and expenses in a structured format.

The application is designed to make it easier to understand how money enters and leaves a user's finances.

3.2 Expense Categorization

To organize transactions into categories so that users can identify the purpose of individual expenses.

Examples include food, transportation, shopping, entertainment, bills, education, and other personal expenses.

3.3 Monthly Expense Monitoring

To provide a monthly overview of financial transactions and help users understand their spending patterns over time.

3.4 Split Bill Management

To enable users to record shared expenses and divide their costs equally among participating individuals.

3.5 Settlement Tracking

To maintain records of payments made to settle shared expenses and show whether the recorded obligations have been settled.

3.6 User-Friendly Interface

To develop a clean, light-themed, responsive interface that is easy to navigate.

3.7 Data Management

To support storing, retrieving, updating, and deleting financial records.

The original frontend uses browser storage. The full-stack version introduces a backend and MySQL database for persistent storage.

3.8 Full-Stack Development

To extend the frontend application with a Java Spring Boot backend and MySQL database.

This introduces practical experience with REST APIs, database integration, backend validation, and layered application architecture.

3.9 Reliable Project Demonstration

To create an application whose functionality can be demonstrated clearly in a college project presentation.

The implementation prioritizes understandable workflows and dependable core features over unnecessary complexity.

---

4. TECHNOLOGY STACK

The project uses a combination of frontend and backend technologies.

4.1 HTML5

HTML5 is used to define the structure of the web application.

It provides the elements that organize the user interface, including:

- Navigation sections.
- Dashboard containers.
- Transaction forms.
- Input fields.

- Buttons.
- Tables and transaction lists.
- Expense categories.
- Split-bill forms.
- Settlement records.
- Sections displaying financial summaries.

HTML establishes the structure that CSS styles and JavaScript makes interactive.

4.2 CSS3

CSS3 is responsible for the application's appearance and layout.

It is used to manage:

- Colors and backgrounds.
- Typography.
- Spacing and alignment.
- Buttons and form controls.
- Cards and dashboard sections.
- Transaction lists.
- Responsive layouts.
- Visual hierarchy.
- Hover effects and other interface details.

The project follows a clean, light-themed design approach.

The purpose is to make financial information readable without overwhelming the user with unnecessary visual elements.

4.3 JavaScript

JavaScript provides the frontend's interactive functionality.

It is responsible for processing user actions and updating the interface.

Its responsibilities include:

- Reading form input.
- Adding transactions.
- Editing existing transactions.
- Deleting transactions.
- Calculating income and expenses.

- Updating dashboard summaries.
- Filtering and searching transactions.
- Organizing transaction data.
- Managing split-bill calculations.
- Recording settlement information.
- Saving and retrieving browser-stored data.
- Exporting supported transaction data to CSV.
- Communicating with backend APIs in the full-stack version.

JavaScript connects the HTML interface with the application's underlying data and calculations.

4.4 Java

Java is used for the planned backend implementation.

Unlike JavaScript, which runs the browser interface, Java handles server-side processing.

The Java backend is intended to receive requests from the frontend, validate information, perform business logic, communicate with the database, and return appropriate responses.

4.5 Spring Boot

Spring Boot is used to develop the Java backend.

It simplifies the creation of a standalone web application and REST API server.

Its intended responsibilities include:

- Starting the backend server.
- Defining API endpoints.
- Processing HTTP requests.
- Validating submitted information.
- Executing business logic.
- Communicating with MySQL.
- Returning JSON responses.
- Organizing application components.

Spring Boot separates backend responsibilities from the frontend interface.

This makes the application easier to maintain and extend.

4.6 Spring Data JPA

Spring Data JPA is intended to simplify database operations.

Instead of writing repetitive database-access code for every operation, repository interfaces can provide standard methods for saving, retrieving, updating, and deleting entities.

For example, a transaction repository can be responsible for accessing stored transaction records.

4.7 Hibernate

Hibernate is the object-relational mapping technology commonly used by Spring Data JPA in this project architecture.

It maps Java objects to relational database tables.

For example, a Java transaction object can correspond to a transaction record in MySQL.

This allows the application to work with Java objects while Hibernate manages the underlying database operations.

4.8 MySQL

MySQL is the relational database selected for persistent storage in the full-stack version.

It is intended to store application records such as:

- Financial transactions.
- Split-bill records.
- Participants associated with shared bills.
- Settlement records.

The database allows information to remain available after the browser is closed, unlike a purely browser-storage-based implementation.

Actual persistence depends on completing and verifying the database configuration and backend connection.

4.9 Apache Maven

Maven is the build and dependency management tool for the Java backend.

It uses a pom.xml file to define the project's dependencies and build configuration.

Maven is intended to manage libraries required by Spring Boot, Spring Data JPA, the MySQL connector, and other backend components.

It also provides commands for compiling and running the application.

During development, a setup issue occurred because the mvn command was not available in the development environment.

This issue must be resolved before the backend can be built using the standard Maven command-line workflow.

4.10 Visual Studio Code

Visual Studio Code is the development environment used to work on the project files.

It is suitable for editing HTML, CSS, JavaScript, Java, configuration files, and documentation.

The frontend can be developed and tested separately from the backend, while the full-stack version requires the appropriate Java and database tooling.

---

5. PROJECT ARCHITECTURE

The project evolved from a frontend-based application toward a full-stack architecture.

5.1 Original Frontend Architecture

The original application consists of three main files:

1. index.html
2. style.css
3. app.js

The HTML file defines the interface.

The CSS file controls its appearance.

The JavaScript file handles user interactions, financial calculations, transaction management, and browser storage.

The basic workflow is:

User
↓
HTML Interface
↓
JavaScript Logic
↓
Browser Local Storage
↓
Updated Dashboard

In this architecture, the application can manage data locally in the user's browser.

It does not require a separate backend server for its basic frontend functionality.

However, browser storage does not provide centralized database storage or automatic synchronization between different devices.

5.2 Full-Stack Architecture

The expanded architecture introduces a Java backend and MySQL database.

The intended workflow is:

User
↓
HTML, CSS and JavaScript Frontend
↓
HTTP Requests to REST APIs
↓

Java Spring Boot Backend
↓
Business Logic and Validation
↓
Spring Data JPA / Hibernate
↓
MySQL Database
↓
Backend Response
↓
Updated Frontend Interface

Each layer has a separate responsibility.

The frontend collects and displays information.

The backend processes requests and applies application rules.

The data-access layer communicates with the database.

MySQL stores the information.

The backend then returns the appropriate response to the frontend.

This architecture provides a foundation for maintaining a more organized and extensible application.

Important development status: The full-stack structure and integration approach were prepared, but the complete MySQL-connected workflow still required environment setup and testing. The architecture should not be considered fully verified until the application has been run successfully from the frontend through the backend to the database.

---

6. FRONTEND IMPLEMENTATION

The frontend is the part of the application that users interact with directly.

It was developed using HTML, CSS, and JavaScript.

The interface was designed to make financial management straightforward.

6.1 Dashboard

The dashboard serves as the main overview of the application.

It is intended to provide a convenient place to review financial information without navigating through every transaction individually.

The frontend version includes a dashboard-oriented interface with financial summaries and transaction information.

The main purpose of the dashboard is to help users understand their recorded financial activity quickly.

6.2 Income Management

Income management allows users to record money received.

A typical income record may include:

- Transaction title.
- Amount.
- Date.
- Category.
- Transaction type.

When an income transaction is recorded, the application can use its amount when calculating the user's total income.

This allows the dashboard to reflect changes in the user's financial records.

6.3 Expense Management

Expense management allows users to record money spent.

An expense record may contain:

- Transaction title.
- Amount.
- Date.
- Category.
- Transaction type.

Examples include food purchases, transportation, shopping, education, entertainment, and other daily expenditures.

The application stores the transaction information and uses it when generating financial summaries.

6.4 Editing Transactions

Users may need to correct a transaction after recording it.

For example, a user might enter an incorrect amount or select the wrong category.

The editing functionality allows an existing record to be modified rather than requiring the user to delete it and create an entirely new record.

The updated information should be reflected in the relevant dashboard calculations.

6.5 Deleting Transactions

The application supports deleting transactions that are no longer required or were entered incorrectly.

Removing a transaction should also update the financial summaries that depend on it.

In the browser-storage version, these changes affect the locally stored transaction records.

In the full-stack version, the corresponding backend operation is intended to update the database.

6.6 Expense Categories

Categories help organize financial transactions.

Instead of displaying every expense as an unrelated entry, the application associates transactions with descriptive categories.

This makes it easier to search for particular transactions and understand the purpose of spending.

6.7 Search and Filtering

The frontend includes search and filtering functionality for transaction records.

These features help users find relevant entries without manually reviewing the entire transaction list.

Search is useful when a user remembers a transaction's name or description.

Filtering is useful when the user wants to narrow the displayed records according to supported criteria.

The exact filtering options depend on the implemented interface.

6.8 Monthly Tracking

Monthly tracking helps users review their financial activity over a particular month.

The application can use transaction dates to organize records and calculate relevant income and expense totals.

This makes it possible to review spending over time rather than relying only on an overall total.

6.9 CSV Export

The frontend version includes CSV export functionality.

CSV stands for Comma-Separated Values.

It is a widely supported format that can be opened in spreadsheet applications.

The purpose of this feature is to make transaction data easier to preserve, review, or analyze outside the application.

The export process converts supported transaction information into a downloadable file.

6.10 Responsive Design

The frontend was designed around a clean, light-themed layout with responsiveness as a design goal.

Responsive design allows the interface to adapt to different screen sizes.

This is important because financial applications may be used on desktop computers, laptops, tablets, and mobile devices.

The final responsive behavior should be checked at different screen sizes before the project is considered complete.

---

7. SPLIT BILL MANAGEMENT SYSTEM

Split bill management is one of the most important additional features introduced during the development of PaisaTrack.

The feature addresses the problem of managing expenses shared by multiple people.

For example, friends may share a restaurant bill, a travel expense, or the cost of a group activity.

Without a proper record, it can become difficult to remember each person's share and determine whether everyone has paid.

The split-bill functionality is designed to make this process easier.

7.1 Purpose of Split Bill Management

The purpose of the system is to:

- Record a shared expense.
- Identify the people participating in that expense.
- Divide the expense equally.
- Calculate the amount attributable to each participant.
- Maintain a record of the shared bill.
- Record settlements.
- Maintain settlement history.

The system is designed for straightforward equal splitting rather than complex financial arrangements.

7.2 Creating a Shared Bill

A user can create a shared-bill record by providing the relevant expense information.

A typical record includes:

- Bill description.
- Total bill amount.
- Participants' names.
- Relevant date or other supported details.

For example, a user might create a shared expense titled "Dinner with Friends."

The total amount is then divided among the participants.

7.3 Equal Bill Splitting

The agreed requirement is to divide shared expenses equally.

The basic formula is:

Individual Share = Total Bill Amount ÷ Number of Participants

Consider the following example.

Total restaurant bill: ₹2,000

Number of participants: 4

Individual share:

₹2,000 ÷ 4 = ₹500

Each participant is responsible for ₹500.

If the application supports different numbers of participants, the calculation changes automatically according to the number included in the shared expense.

For real financial transactions, any rounding remainder should be handled explicitly so that the allocated shares add up to the original bill amount.

7.4 Adding Participants by Name

The project requirements specify that participants can be identified using their names.

Creating a separate user account for every participant is not required for the intended split-bill workflow.

For example, the participants could be:

- Prayag.
- Yashvardhan.
- Rohit.
- Aman.

The application can associate these names with the shared expense.

This keeps the workflow simple for a college project or a small group of friends.

However, names alone do not authenticate participants or establish that they personally accessed the application.

7.5 Multiple Groups

The split-bill system is designed to support multiple groups.

Different groups may represent different sets of people or different shared-expense activities.

For example:

Group 1: College Friends

Members participate in shared meals and college outings.

Group 2: Travel Group

Members share transportation, accommodation, and other travel expenses.

Group 3: Family

Family members maintain records of shared purchases or activities.

Keeping groups separate helps organize expenses and reduces confusion between unrelated activities.

The exact group-management interface should be verified in the running application.

7.6 Personal Share Accounting

A specific requirement of the project is that the user's own share of a shared expense should count as a personal expense.

Consider a bill of ₹2,000 divided equally among four people.

Each person's share is ₹500.

The user's personal expense attributable to that bill is ₹500, not ₹2,000.

This distinction is important because the entire shared bill should not automatically be treated as the user's personal expenditure.

The personal expense calculation should count the user's allocated share.

Accounting principle:

Personal Expense from Shared Bill = User's Allocated Share

The full bill can still be maintained in the split-bill records for group calculations.

This approach helps avoid overstating personal spending.

7.7 Settlement Tracking

A settlement is a recorded payment made to resolve an outstanding obligation associated with a shared expense.

The application is designed to record settlement information so users can review whether shared expenses have been settled.

A settlement record may contain:

- Relevant shared bill.
- Person making the payment.
- Person receiving the payment, where applicable.
- Settlement amount.
- Settlement date.
- Settlement status or related history.

The exact fields depend on the implementation.

For example, if one participant owes another participant ₹500, a settlement record can document the payment and update the recorded outstanding amount.

7.8 Complete Settlements

The agreed requirements specify complete settlements rather than partial payments.

This means a participant settles the entire outstanding amount in one recorded settlement.

The application is not intended to manage multiple partial payments against the same outstanding obligation.

For example:

Outstanding amount: ₹500

Recorded full settlement: ₹500

Remaining amount: ₹0

The obligation can then be treated as settled in the relevant records.

The system should prevent a full-settlement action from being recorded for an amount that does not match the applicable outstanding obligation.

7.9 Settlement History

The project requirements include maintaining a history of settlements.

This allows the user to review previous settlement activity instead of relying on memory.

A useful settlement history can show:

- Which shared bill was involved.
- Who paid.
- Who received the payment, where applicable.
- The amount settled.
- When the settlement was recorded.

Maintaining this history improves transparency and helps users resolve confusion about past payments.

7.10 No UPI Integration

The project requirements explicitly exclude UPI payment integration.

The application is intended to record and manage settlement information rather than directly initiate digital payments.

A settlement record should therefore not be interpreted as proof that money was transferred through a banking application.

Actual payments must be completed separately.

7.11 No Mandatory Participant Accounts

The project requirements also exclude mandatory accounts or profiles for every person participating in a shared bill.

This keeps the system simple.

The application can record participants by name without requiring each person to register.

However, this design does not provide independent authentication for those participants.

7.12 Split-Bill Reports

The planned reporting functionality includes monthly reports and settlement reports.

Monthly reports can help users review relevant expense activity over time.

Settlement reports can help users review recorded settlement information.

These reports should be based on the underlying transaction and settlement records rather than separate, manually maintained totals.

The correctness of the reports depends on the implementation and testing of the associated calculations.

---

8. BUSINESS LOGIC AND FINANCIAL CALCULATIONS

Financial calculations are central to the reliability of the application.

The application must distinguish between income, personal expenses, shared bills, and settlements.

8.1 Total Income

Total income is calculated by adding the amounts of valid income transactions included in the selected reporting period.

Formula:

Total Income = Sum of Recorded Income Transactions

8.2 Total Personal Expenses

Total personal expenses represent the user's recorded personal expenditure.

This includes ordinary personal expense transactions and the user's allocated share of shared bills where applicable.

Formula:

Total Personal Expenses = Ordinary Personal Expenses + User's Shares of Shared Bills

The calculation must avoid counting the full group bill as the user's expense when only the user's allocated share belongs in personal spending.

8.3 Net Balance

A basic net balance can be calculated as:

Net Balance = Total Income − Total Personal Expenses

This figure represents the difference between the income and expenses included in the calculation.

It should not automatically be interpreted as the user's actual bank balance because the application does not necessarily track every account, cash balance, or external financial obligation.

8.4 Individual Share

Individual Share = Total Shared Bill ÷ Number of Participants

For bills that do not divide evenly to the smallest currency unit, the application must apply a consistent rounding rule.

8.5 Outstanding Obligations

For a participant, the outstanding amount represents the amount still owed under the application's shared-expense records.

Conceptually:

Outstanding Amount = Recorded Amount Owed − Recorded Settlements Applied to That Obligation

Because the project requirements exclude partial settlements, a full settlement should resolve the applicable outstanding obligation.

The exact direction of the obligation matters: the person who owes money and the person who is owed money must be identified correctly.

8.6 Avoiding Double Counting

The application must ensure that recording a shared bill does not cause the user's personal expenses to increase by the entire bill when the user is responsible for only a fraction.

Similarly, recording a settlement should not incorrectly add the same amount to personal expenses again.

A settlement resolves a recorded obligation. It is not automatically a new expense of the same amount.

This distinction is important for accurate reporting.

---

9. BROWSER STORAGE AND DATA PERSISTENCE

9.1 Local Storage in the Original Version

The original frontend uses browser Local Storage to retain application data.

Local Storage is a browser feature that stores key-value pairs on the user's device.

JavaScript can use it to save application data and retrieve it when the page is reopened.

This allows the frontend to maintain transaction records, shared-bill information, and settlement history without a separate database server.

9.2 Advantages of Local Storage

The original approach offers several advantages:

- Simple implementation.
- No separate backend required.
- No database server required.
- Convenient frontend testing.
- Suitable for demonstrating basic application workflows.
- Data can remain available after the browser is closed.

9.3 Limitations of Local Storage

Local Storage also has important limitations.

The stored information is associated with the browser and origin.

It is not automatically synchronized between devices.

Clearing the relevant browser storage may remove the application's saved data.

It does not provide the same centralized database management capabilities as MySQL.

It also does not provide server-side validation or secure user authentication.

For these reasons, the project was extended toward a backend-and-database architecture.

9.4 Browser Storage Fallback

The full-stack version was prepared with a browser-storage fallback approach.

The intended purpose is to allow the frontend to retain a local operating path when backend connectivity is unavailable.

However, fallback behavior must be tested carefully.

If the browser and MySQL each contain separate copies of the same records, their information can become inconsistent.

A reliable implementation should clearly distinguish between local mode and server-connected mode and should not silently imply that local changes have been saved to MySQL.

---

10. SPRING BOOT BACKEND

The backend was planned to extend the frontend into a full-stack application.

Its purpose is to handle requests from the frontend and communicate with the MySQL database.

10.1 REST API Architecture

REST APIs allow the frontend and backend to communicate over HTTP.

The frontend sends requests containing the required information.

The backend processes these requests and returns responses, commonly in JSON format.

For example, when a user submits a new expense, the frontend can send the expense details to the backend.

The backend validates the information, saves it through the data-access layer, and returns a response indicating the result.

10.2 Transaction API

The planned transaction API is responsible for personal financial records.

Typical operations include:

- Creating a transaction.
- Retrieving transaction records.
- Retrieving a particular transaction.
- Updating a transaction.
- Deleting a transaction.

A conventional REST design might use endpoints such as:

POST /api/transactions

GET /api/transactions

GET /api/transactions/{id}

PUT /api/transactions/{id}

DELETE /api/transactions/{id}

These paths describe the intended API design and should not be assumed to match every endpoint in the prepared source code without verification.

10.3 Split-Bill API

The split-bill API is intended to manage shared expenses.

Its responsibilities include:

- Creating shared bills.
- Retrieving shared-bill records.
- Associating participants with bills.
- Storing relevant bill information.
- Supporting updates where required.
- Managing the information needed for settlement calculations.

Possible endpoint patterns include:

POST /api/split-bills

GET /api/split-bills

GET /api/split-bills/{id}

These are representative endpoint patterns, not a verified list of all implemented routes.

10.4 Settlement API

The settlement API is intended to record and retrieve settlement information.

Its responsibilities include:

- Recording a full settlement.
- Associating the settlement with the appropriate bill or obligation.
- Retrieving settlement history.
- Updating the recorded settlement state when appropriate.

A representative endpoint might be:

POST /api/settlements

The exact endpoint and request structure must be confirmed from the backend source code.

10.5 Validation

Backend validation is important because frontend validation alone is not sufficient.

A user can potentially bypass browser-side checks by sending a request directly to the backend.

The backend should therefore validate submitted data.

Examples include:

- Amounts must be valid monetary values.
- Required fields must be present.
- A shared bill must have a valid total amount.
- A bill must have at least one participant.
- A settlement must refer to a valid outstanding obligation.
- Invalid operations should return suitable error responses.

Validation requirements should be enforced on the server before data is saved.

10.6 Layered Architecture

A layered backend separates different responsibilities.

A typical structure includes:

Controller Layer

Receives HTTP requests and returns HTTP responses.

Service Layer

Implements business rules, calculations, and validation that involve application logic.

Repository Layer

Provides database-access operations.

Entity Layer

Defines the Java objects mapped to database records.

Configuration Layer

Contains application settings, database configuration, and other environment-specific options.

Separating these responsibilities improves readability, maintainability, and testing.

The precise package structure must be verified against the actual project files.

---

11. MYSQL DATABASE DESIGN

MySQL was selected as the database for the full-stack version.

The intended database structure supports transaction records, shared bills, participants, and settlements.

The following is a conceptual description of the data model.

11.1 Transactions Table

The transactions table is intended to store individual income and expense records.

Potential fields include:

- Transaction ID.
- Description.
- Amount.
- Transaction type.
- Category.
- Transaction date.
- Creation timestamp.

The transaction ID uniquely identifies each record.

The transaction type distinguishes income from expenses.

The category describes the purpose of the transaction.

The date allows transactions to be organized into reporting periods.

11.2 Split Bills Table

The split-bills table is intended to store information about shared expenses.

Potential fields include:

- Split-bill ID.
- Bill description.
- Total amount.
- Bill date.
- Creation timestamp.

The bill ID identifies the shared expense.

The total amount represents the original group bill.

11.3 Participants Table

A participant structure is needed to associate people with shared expenses.

Depending on the chosen implementation, participant records may contain:

- Participant ID.
- Participant name.
- Associated shared-bill ID.
- Calculated share or other relevant details.

A normalized design can also separate reusable participant records from the relationships between participants and shared bills.

Because the project does not require independent participant accounts, these records should not be confused with authenticated application users.

11.4 Settlements Table

The settlements table is intended to record completed settlement actions.

Potential fields include:

- Settlement ID.
- Associated bill or obligation.
- Payer.
- Recipient, where applicable.
- Settlement amount.
- Settlement date.
- Settlement status.

The relationships between these records must be designed carefully so that settlement history remains associated with the correct financial obligation.

11.5 Database Relationships

The conceptual relationships are:

- A shared bill can have multiple participants.
- A shared bill can have associated settlement records.
- A participant can appear in multiple shared bills.
- A settlement belongs to the relevant shared-expense obligation.
- Personal transactions are stored separately from shared-bill records, even when a shared bill contributes to the user's personal expense total.

The last distinction helps avoid double counting.

The exact database schema and foreign-key relationships must be confirmed from the SQL file and entity classes in the project.

11.6 MySQL Server and MySQL Workbench

During setup, an issue occurred because MySQL Workbench was installed but no local server instance was available.

MySQL Workbench is a graphical tool for working with MySQL databases.

It is not itself the MySQL database server.

The application requires an actual MySQL Server installation or access to another running MySQL instance.

For a local development setup, the necessary steps are:

1. Install MySQL Server.
2. Configure the database server.
3. Set up the database credentials.
4. Start the MySQL service.
5. Create the project database.
6. Configure the backend connection.
7. Verify that Spring Boot can connect to the database.

The missing local instance was therefore a database-environment setup issue, not necessarily a problem with the frontend code.

---

12. MAVEN CONFIGURATION

Maven is an important part of the Java backend setup.

It is used to manage project dependencies and build the Spring Boot application.

The project requires a compatible Java development kit and Maven installation.

12.1 Java Requirement

The backend was planned around Java 17 or a compatible supported Java version, depending on the Spring Boot version specified in the project configuration.

The installed Java version must match the requirements of the backend project.

12.2 Maven Requirement

The project uses a Maven configuration file named:

pom.xml

This file describes the project's build configuration and dependencies.

A standard Maven setup allows developers to compile and run the backend without manually downloading every library.

12.3 Development Issue

During development, the command:

mvn

was reported as unavailable.

This indicates that Maven was not installed correctly, was not included in the system PATH, or was otherwise not accessible through the terminal.

The exact cause must be confirmed on the development computer.

12.4 Required Verification

The following commands can be used in a terminal after installing and configuring the relevant tools:

java -version

mvn -version

If the commands return valid version information, the development environment can recognize Java and Maven.

From the backend project's root directory, the following command can be used to build the application:

mvn clean package

After a successful build, the application can be started using the project's configured Spring Boot workflow.

For example:

mvn spring-boot:run

These commands assume that the project contains a valid Maven configuration and the necessary Spring Boot plugin.

The backend must be tested after the environment has been configured.

---

13. FRONTEND AND BACKEND INTEGRATION

Connecting the frontend to the backend is necessary for the full-stack version.

The frontend must send requests to the backend rather than relying exclusively on Local Storage for every operation.

13.1 Request Flow

Consider the process of creating an expense.

Step 1: The user opens the application.

Step 2: The user enters the transaction details in the frontend form.

Step 3: JavaScript validates the input for immediate feedback.

Step 4: The frontend sends an HTTP request to the Spring Boot backend.

Step 5: The backend validates the submitted data.

Step 6: The backend processes the request.

Step 7: Spring Data JPA and Hibernate perform the required database operation.

Step 8: MySQL stores the transaction.

Step 9: The backend returns a response.

Step 10: The frontend updates the transaction list and relevant financial summaries.

This workflow allows the application to use the database as its persistent data source.

13.2 Backend Connectivity

For local development, the frontend and backend may run on different local ports.

The frontend must send requests to the correct backend address.

If the frontend is opened through a local development server, the backend address must be configured appropriately.

Cross-Origin Resource Sharing, commonly called CORS, may also need configuration when the frontend and backend use different origins.

The exact configuration depends on how the project is launched.

13.3 Database Connection

The backend must use the correct database connection settings.

These normally include:

- Database URL.
- Database name.
- Database username.
- Database password.
- Appropriate MySQL driver and JPA configuration.

Credentials should be managed carefully and should not be committed to a public repository.

13.4 Data Consistency

The frontend and backend must agree on the structure of transaction, split-bill, and settlement data.

For example, if the frontend expects a transaction amount to be returned as a number, the backend should return a compatible value.

The same principle applies to dates, identifiers, transaction types, and settlement statuses.

A mismatch between the frontend and backend data structures can cause features to fail even when both components run independently.

---

14. CSV EXPORT AND REPORTING

CSV export is included in the frontend functionality.

It provides a way to take transaction information outside the application.

A typical CSV file may contain columns such as:

- Description.
- Amount.
- Type.
- Category.
- Date.

The exported data can be opened in spreadsheet software for further review.

The reporting requirements also include monthly expense reporting and settlement reporting.

These reports are intended to help users understand their financial activity and review shared-expense obligations.

A full-stack implementation should ensure that the reporting calculations use the appropriate records and do not double-count shared expenses or settlements.

CSV export should also be tested with empty data, special characters, and different transaction categories.

---

15. PROJECT REQUIREMENTS AND DESIGN DECISIONS

The following decisions were established during project planning.

15.1 Included Requirements

The application is intended to support:

- Personal income and expense tracking.
- Transaction editing and deletion.
- Expense categorization.
- Monthly expense monitoring.
- Transaction search and filtering.
- CSV export.
- Equal splitting of shared bills.
- Multiple groups.
- Participants identified by name.
- Settlement tracking.
- Full settlements rather than partial payments.
- Settlement history.
- Monthly and settlement reports.
- Recording the user's own share as a personal expense.
- A clean, light-themed interface.
- Responsive frontend design.
- Browser-based data storage in the original version.
- Java Spring Boot backend integration.
- MySQL database integration.

The frontend functionality was developed first, and the full-stack requirements were prepared as an extension.

The completeness of individual features should be verified against the final source code and through functional testing.

15.2 Explicitly Excluded Features

The agreed scope excludes:

Budget Management

The project is not intended to include a separate budget-setting system.

Savings Goals

The project does not require a dedicated savings-goal tracking feature.

UPI Integration

The application records settlement information rather than initiating UPI payments.

Mandatory Accounts for Every Participant

Participants in shared expenses can be represented by name without requiring each person to register.

Partial Settlements

The intended workflow records full settlements rather than a sequence of partial payments against an obligation.

These exclusions keep the project focused on its principal purpose: personal expense management and straightforward bill sharing.

---

16. PROJECT DEVELOPMENT HISTORY

The project evolved through multiple stages.

Stage 1: Initial Project Reference

The development began with an existing expense-tracker project used as a reference.

The original reference repository was:

ManikMaity/Expense-Tracker-App

The repository was used as a starting reference for the frontend-oriented expense-tracking application.

The project subsequently evolved toward the requirements of BillBuddy/PaisaTrack.

Stage 2: Frontend Development

The next stage focused on the user interface and frontend functionality.

The principal files included:

- index.html
- style.css
- app.js
- README.md

The frontend was designed to support income and expense records, transaction management, dashboard summaries, monthly tracking, searching, filtering, and CSV export.

Browser Local Storage was used to retain transaction and split-bill information.

Stage 3: Split-Bill Requirements

The project was expanded to include shared-expense management.

The requirements were defined around equal splitting, multiple groups, name-based participants, full settlements, settlement history, and personal-share accounting.

The goal was to make bill sharing part of the same financial management application rather than a separate tool.

Stage 4: Full-Stack Expansion

The next stage introduced the planned Java Spring Boot backend and MySQL database.

The prepared full-stack project included the frontend and backend integration approach, REST API functionality, Spring Data JPA, database configuration, and SQL-related resources.

A browser-storage fallback was also part of the prepared approach.

The purpose of this stage was to transition from a browser-only application toward a database-backed system.

Stage 5: Environment Setup

The development process then moved to setting up the required tools.

MySQL Workbench was installed, but a local MySQL Server instance was not available.

It was established that Workbench alone does not provide the database server.

The Maven command was also reported as unavailable.

These setup issues prevented the complete full-stack workflow from being considered verified.

Stage 6: Current Development Status

The frontend is the established foundation of the project.

The full-stack architecture has been prepared, but the remaining work includes completing the local Java/Maven/MySQL setup, verifying backend compilation, checking database connectivity, and testing the complete application workflow.

The next development milestone is to run the full application successfully and verify that frontend operations are persisted correctly in MySQL.

---

17. TESTING AND QUALITY ASSURANCE

Testing is necessary to verify that the application works correctly.

The following test cases should be used to validate the final implementation.

17.1 Frontend Testing

Test 1: Add Income

Enter a valid income transaction and verify that it appears in the transaction list and affects the income summary.

Test 2: Add Expense

Enter a valid expense and verify that it appears in the transaction list and affects the expense summary.

Test 3: Edit Transaction

Change the amount or description of an existing transaction and verify that the updated information appears correctly.

Test 4: Delete Transaction

Delete an existing transaction and verify that it is removed from the list and excluded from the relevant calculations.

Test 5: Search Transactions

Search for a known transaction and verify that the appropriate record is displayed.

Test 6: Monthly Tracking

Create transactions with relevant dates and verify that the monthly totals include the correct records.

Test 7: CSV Export

Export transaction data and verify that the generated file contains the expected information.

17.2 Split-Bill Testing

Test 8: Equal Splitting

Create a bill of ₹2,000 with four participants.

Verify that each participant's share is ₹500.

Test 9: Multiple Groups

Create separate groups and verify that their shared-expense records remain correctly associated with the relevant group.

Test 10: Participant Management

Add participants by name and verify that the names are associated with the appropriate shared bill.

Test 11: Personal Expense Calculation

Create a shared bill and verify that the user's personal expense calculation includes the user's share rather than the entire group bill.

Test 12: Full Settlement

Record a valid full settlement and verify that the applicable outstanding obligation becomes settled.

Test 13: Settlement History

Verify that a recorded settlement appears in the relevant history with the correct information.

Test 14: Invalid Settlement

Attempt to record a settlement for an invalid amount or nonexistent obligation and verify that the application rejects it appropriately.

17.3 Backend Testing

Test 15: Backend Startup

Start the Spring Boot application and verify that it starts without configuration or dependency errors.

Test 16: Database Connectivity

Verify that Spring Boot successfully connects to the configured MySQL database.

Test 17: Create Transaction Through API

Submit a valid transaction request and verify that the backend returns an appropriate response and the record is stored in MySQL.

Test 18: Retrieve Transaction Through API

Request transaction records and verify that the backend returns the expected data.

Test 19: Update and Delete Operations

Verify that supported update and delete requests produce the correct database changes.

Test 20: Split-Bill Persistence

Create a shared bill through the backend and verify that the relevant bill and participant information is stored correctly.

Test 21: Settlement Persistence

Record a settlement and verify that the relevant settlement history remains available after the backend is restarted.

17.4 Full-Stack Integration Testing

The final test is to run the frontend, backend, and database together.

The tester should:

1. Start MySQL Server.
2. Verify the project database exists.
3. Start the Spring Boot backend.
4. Launch the frontend.
5. Create an income transaction.
6. Create a personal expense.
7. Create a shared bill.
8. Verify the participant shares.
9. Record a full settlement.

10. Restart the backend.
11. Retrieve the saved records.
12. Verify that the data remains available.

Successful completion of this workflow would demonstrate that the frontend, backend, and database are communicating correctly.

Testing status: These are the required verification procedures. They should not be described as successfully completed until the corresponding tests have actually been performed.

---

18. CURRENT LIMITATIONS

The project has a defined scope and several development limitations.

18.1 Browser Storage Limitations

The original frontend relies on browser Local Storage.

This approach is useful for a simple frontend application but does not provide centralized, multi-device data synchronization.

18.2 Backend Environment Setup

The reported Maven installation issue must be resolved before the normal backend build workflow can be verified.

18.3 MySQL Server Configuration

A MySQL Server instance must be installed or otherwise made available and configured.

Installing MySQL Workbench alone is insufficient.

18.4 Integration Verification

The full-stack application must be tested to confirm that transactions, shared bills, and settlements are correctly persisted in MySQL.

18.5 Authentication

The agreed requirements do not include a complete authentication system.

Name-based participants in split bills should not be treated as authenticated users.

18.6 Direct Payments

The application does not initiate UPI payments.

It manages settlement records rather than transferring money.

18.7 Production Readiness

The application is being developed as an academic project.

Before production use, additional work may be required for authentication, authorization, deployment security, data backup, error monitoring, concurrency handling, and other operational requirements.

---

19. FUTURE SCOPE

Although the project is focused on its agreed core requirements, it can be extended in the future.

Possible improvements include:

- Secure user registration and login.
- Multi-device synchronization.
- Cloud deployment.
- Improved financial visualizations.
- Additional report formats.
- Automated database backups.
- More detailed transaction analytics.
- Optional payment-service integration.
- More advanced shared-expense allocation methods.
- Improved accessibility.
- Automated unit and integration tests.
- Enhanced database security and error handling.

These are possible future improvements, not claims about the current implementation.

The core project should first be completed and tested before additional features are introduced.

---

20. EXPECTED PROJECT OUTCOME

The expected outcome is a functional web application that brings personal expense tracking and shared-bill management together.

Users should be able to record income and expenses, organize transactions, review monthly spending, and export supported financial data.

The split-bill system should make it easier to divide expenses among friends, calculate individual shares, record full settlements, and maintain settlement history.

The full-stack implementation should provide a Java Spring Boot backend connected to a MySQL database.

The completed application should demonstrate how a web frontend communicates with backend APIs and how structured financial information can be stored and retrieved from a relational database.

It should also demonstrate practical software development concepts, including frontend development, backend architecture, database design, REST API communication, business logic, and testing.

---

21. LEARNING OUTCOMES

The project provides practical experience in several areas of software development.

Frontend Development

Understanding how HTML, CSS, and JavaScript work together to create an interactive web application.

Data Handling

Learning how to create, update, retrieve, and delete financial records.

Browser Storage

Understanding how Local Storage can be used to retain information in a browser.

Financial Logic

Implementing calculations involving income, expenses, shared costs, and settlements.

Backend Development

Understanding how Java and Spring Boot can be used to develop server-side application functionality.

REST APIs

Learning how a frontend communicates with a backend through HTTP requests and responses.

Relational Databases

Understanding how MySQL stores structured information and how tables relate to one another.

ORM Technology

Learning how Spring Data JPA and Hibernate can simplify database operations.

Build Management

Understanding how Maven manages dependencies and builds a Java application.

Software Testing

Learning how to verify individual features and test the complete application workflow.

These skills provide a foundation for developing more advanced full-stack applications in the future.

22. DEVELOPER INFORMATION

Project Developer: PRAYAG SHARMA

PRAYAG SHARMA is the developer responsible for the BillBuddy/PaisaTrack project.

The project focuses on developing a practical financial management system that combines personal expense tracking with shared-bill management.

The development work includes frontend implementation, financial calculations, transaction management, browser-based data persistence, and the planned integration of a Java Spring Boot backend with a MySQL database.

The project demonstrates the application of web development, backend programming, database management, and software engineering principles to a practical everyday problem.

---

23. CONCLUSION

BillBuddy/PaisaTrack is a personal finance and shared-expense management application designed to simplify the way users record their financial transactions and manage bills shared with friends.

The project began with a frontend implementation using HTML, CSS, and JavaScript. The initial application provided the foundation for transaction management, financial summaries, monthly tracking, searching, filtering, CSV export, and browser-based data storage.

The project was subsequently expanded to include a split-bill management system. Its requirements include equal expense division, multiple groups, name-based participants, full settlements, settlement history, and correct accounting of the user's own share of a shared expense.

The next major stage involves completing the Java Spring Boot backend and MySQL integration. This requires resolving the development environment issues, configuring MySQL Server, making Maven available, and verifying that the frontend can communicate with the backend and persist information in the database.

Once the full-stack workflow has been tested successfully, the application will provide a more complete demonstration of personal financial management, shared-expense calculations, REST API development, and relational database integration.

BillBuddy/PaisaTrack represents the development of a practical financial management solution by PRAYAG SHARMA, combining frontend web technologies with a planned Java and MySQL backend to address everyday expense-tracking and bill-sharing problems.